Secciones

- **Entornos de desarrollo recomendados**
- **Configuración de VS Code para C**
    - **Entorno Windows**
    - **Entorno Linux**
    - **Entorno Mac**
- **Debug en VSCode**
- **Compilación y ejecución manual**

Entornos de desarrollo recomendados

Para dar los primeros pasos se recomienda fuertemente el uso de algún IDE, ya que permite una ejecución paso a paso y provee herramientas para *debugging* de los programas.

Los entornos de desarrollo recomendados por la cátedra son:

- **Visual Studio Code** - (https://code.visualstudio.com) con plugin para C y compilador instalado, varía según entorno win, linux o mac.
- **CLion** - (https://www.jetbrains.com/clion) es de licencia paga, pero gratuito para licencias de estudiante con dominio de correo @fi.uba.ar
- **Code::Blocks** - (https://www.codeblocks.org)
- **Dev-C++** - (https://www.embarcadero.com/free-tools/dev-cpp)
- **GDB Online** - (https://www.onlinegdb.com)

Configuración de VS Code para C

Para todos los entornos el primer paso es descargar **Visual Studio Code** desde https://code.visualstudio.com/download

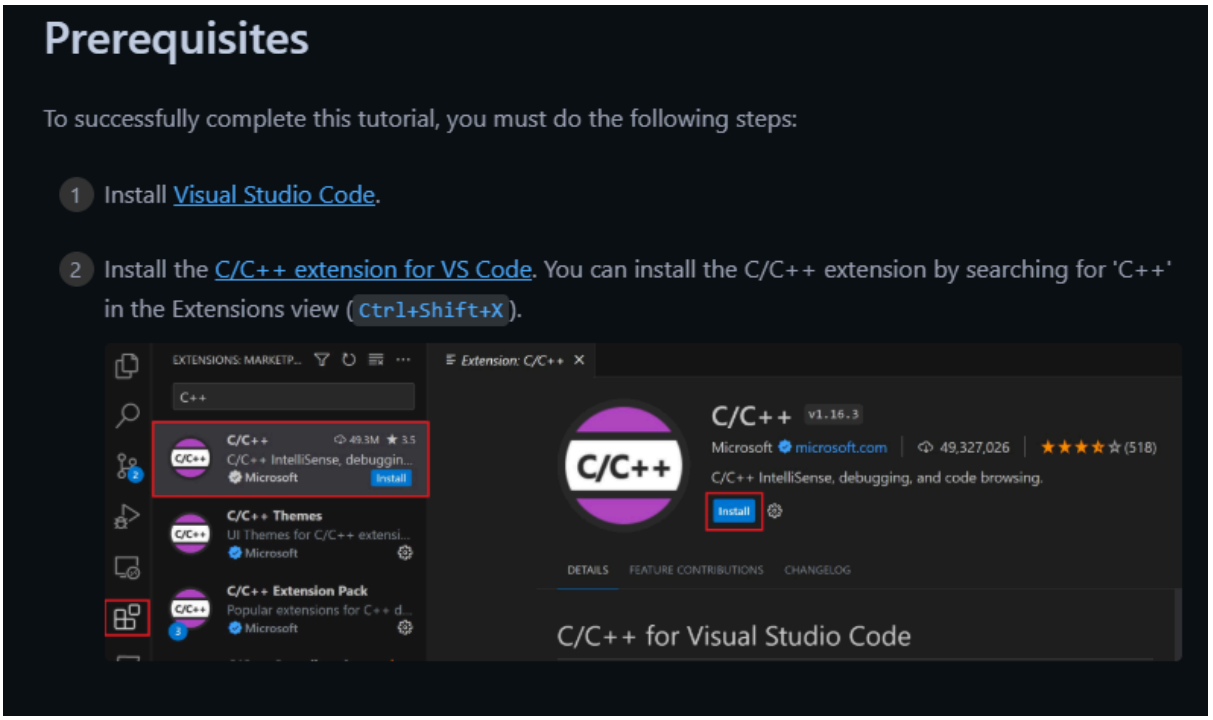
Entorno Windows

Tome la siguiente guía de referencia y siga los pasos descritos a continuación.

Guía: https://code.visualstudio.com/docs/cpp/config-mingw

Instalar **VSCode** a través del link del paso 1

Instalar la extensión **"C/C++ extension for VS Code"** dentro de VSCode



Instalar las herramientas provistas por el instalador **“MinGW-w64 toolchain”** a través del link **“direct link to the installer”**

Installing the MinGW-w64 toolchain

Get the latest version of MinGW-w64 via MSYS2, which provides up-to-date native builds of GCC, MinGW-w64, and other helpful C++ tools and libraries. This will provide you with the necessary tools to compile your code, debug it, and configure it to work with IntelliSense.

To install the MinGW-w64 toolchain, check out this video or follow the steps below:

1 You can download the latest installer from the MSYS2 page or use this direct link to the installer.

Luego de realizar los pasos indicados durante la instalación, al finalizar se abrirá una consola de comando, ahí se debe ejecutar el comando:

4 In this terminal, install the MinGW-w64 toolchain by running the following command:

Shell

```
pacman -S --needed base-devel mingw-w64-ucrt-x86_64-toolchain
```

5 Accept the default number of packages in the `toolchain` group by pressing `Enter`.

```
$ pacman -S --needed base-devel mingw-w64-ucrt-x86_64-toolchain
warning: base-devel-2022.12-2 is up to date -- skipping
:: There are 19 members in group mingw-w64-ucrt-x86_64-toolchain:
:: Repository ucrt64
   1) mingw-w64-ucrt-x86_64-binutils  2) mingw-w64-ucrt-x86_64-crt-git
   3) mingw-w64-ucrt-x86_64-gcc  4) mingw-w64-ucrt-x86_64-gcc-ada
   5) mingw-w64-ucrt-x86_64-gcc-fortran  6) mingw-w64-ucrt-x86_64-gcc-libgfortran
   7) mingw-w64-ucrt-x86_64-gcc-libs  8) mingw-w64-ucrt-x86_64-gcc-objc
   9) mingw-w64-ucrt-x86_64-gdb  10) mingw-w64-ucrt-x86_64-gdb-multiarch
   11) mingw-w64-ucrt-x86_64-headers-git  12) mingw-w64-ucrt-x86_64-libgccjit
   13) mingw-w64-ucrt-x86_64-libmangle-git  14) mingw-w64-ucrt-x86_64-libwinpthread-git
   15) mingw-w64-ucrt-x86_64-make  16) mingw-w64-ucrt-x86_64-pkgconf
   17) mingw-w64-ucrt-x86_64-tools-git  18) mingw-w64-ucrt-x86_64-winpthreads-git
   19) mingw-w64-ucrt-x86_64-winstorecompat-git

Enter a selection (default=all):
```

6 Enter `Y` when prompted whether to proceed with the installation.

Luego se debe actualizar el PATH del sistema siguiendo las instrucciones del paso 7

7. Add the path of your MinGW-w64 `bin` folder to the Windows `PATH` environment variable by using the following steps:
   1. In the Windows search bar, type **Settings** to open your Windows Settings.
   2. Search for **Edit environment variables for your account**.
   3. In your **User variables**, select the `Path` variable and then select **Edit**.
   4. Select **New** and add the MinGW-w64 destination folder you recorded during the installation process to the list. If you used the default settings above, then this will be the path: `C:\msys64\ucrt64\bin`.
   5. Select **OK**, and then select **OK** again in the **Environment Variables** window to update the `PATH` environment variable. You have to reopen any console windows for the updated `PATH` environment variable to be available.

Una vez realizado el paso anterior, y verificado que tengamos el compilador y debugger correctamente instalados, en una consola CMD ejecutar individualmente los comandos:

```
gcc --version
gdb --version
```

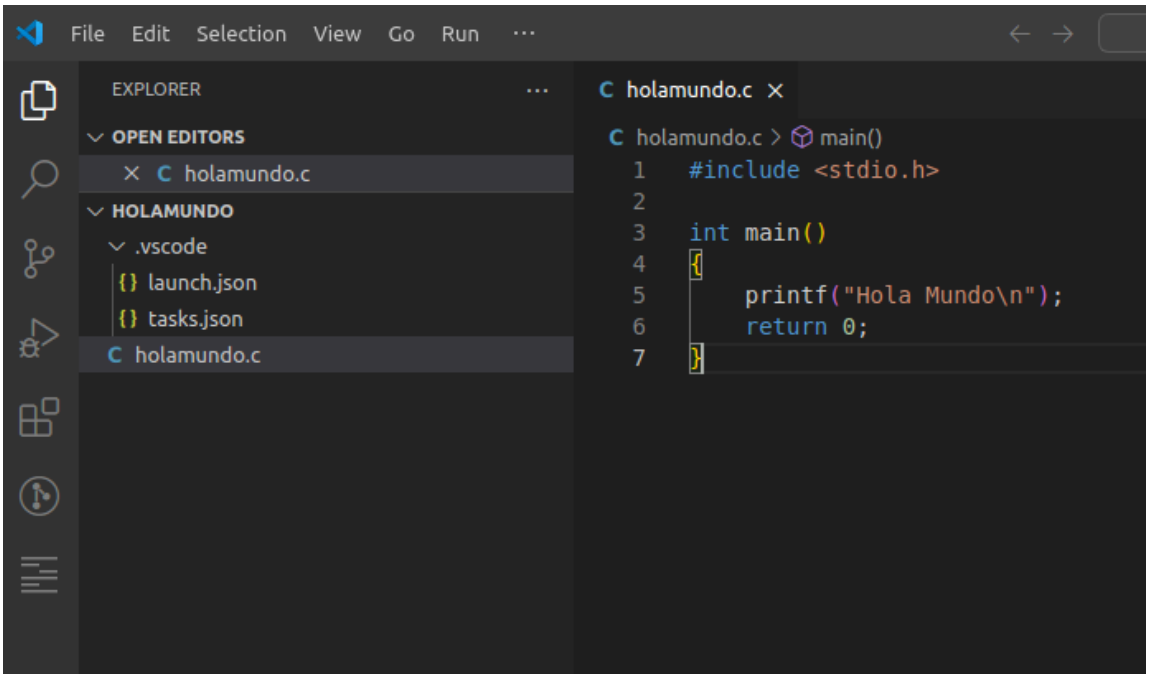
Deberemos observar una salida similar a la siguiente en donde observaremos las versiones instaladas de dichas herramientas

```
Símbolo del sistema
Microsoft Windows [Versión 10.0.19045.6216]
(c) Microsoft Corporation. Todos los derechos reservados.

C:\Users\diego>gcc --version
gcc (Rev2, Built by MSYS2 project) 14.2.0
Copyright (C) 2024 Free Software Foundation, Inc.
This is free software; see the source for copying conditions.  There is NO
warranty; not even for MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE.


C:\Users\diego>gdb --version
GNU gdb (GDB) 15.2
Copyright (C) 2024 Free Software Foundation, Inc.
License GPLv3+: GNU GPL version 3 or later <http://gnu.org/licenses/gpl.html>
This is free software: you are free to change and redistribute it.
There is NO WARRANTY, to the extent permitted by law.
```

Crearemos un programa básico. Para esto configuraremos los archivos de vscode que nos permitirán compilar y ejecutar los programas.
Crearemos dos archivos dentro del directorio **.vscode** dentro de nuestro directorio de trabajo, **launch.json** y **tasks.json**



tasks.json

En este archivo definiremos dos tareas:

"C Build active file" (nos permite generar un ejecutable)
"C Run active file" (nos permite ejecutar el programa)

```
{
    "tasks": [
      {
        "type": "cppbuild",
        "label": "C Build active file",
        "command": "C:\\msys64\\ucrt64\\bin\\gcc.exe",
        "args": [
          "-fdiagnostics-color=always",
          "-g",
          "${file}",
          "-o",
          "${fileDirname}\\${fileBasenameNoExtension}.exe"
        ],
        "options": {
          "cwd": "${fileDirname}"
        },
        "problemMatcher": ["$gcc"],
        "group": {
          "kind": "build",
          "isDefault": true
        },
        "detail": "Task generated by User."
      },
      {
        "type": "shell",
        "label": "C Run active file",
        "command": "${fileDirname}\\${fileBasenameNoExtension}.exe",
        "options": {
          "cwd": "${fileDirname}"
        },
        "detail": "Task generated by User."
      }
    ],
    "version": "2.0.0"
  }
```

launch.json

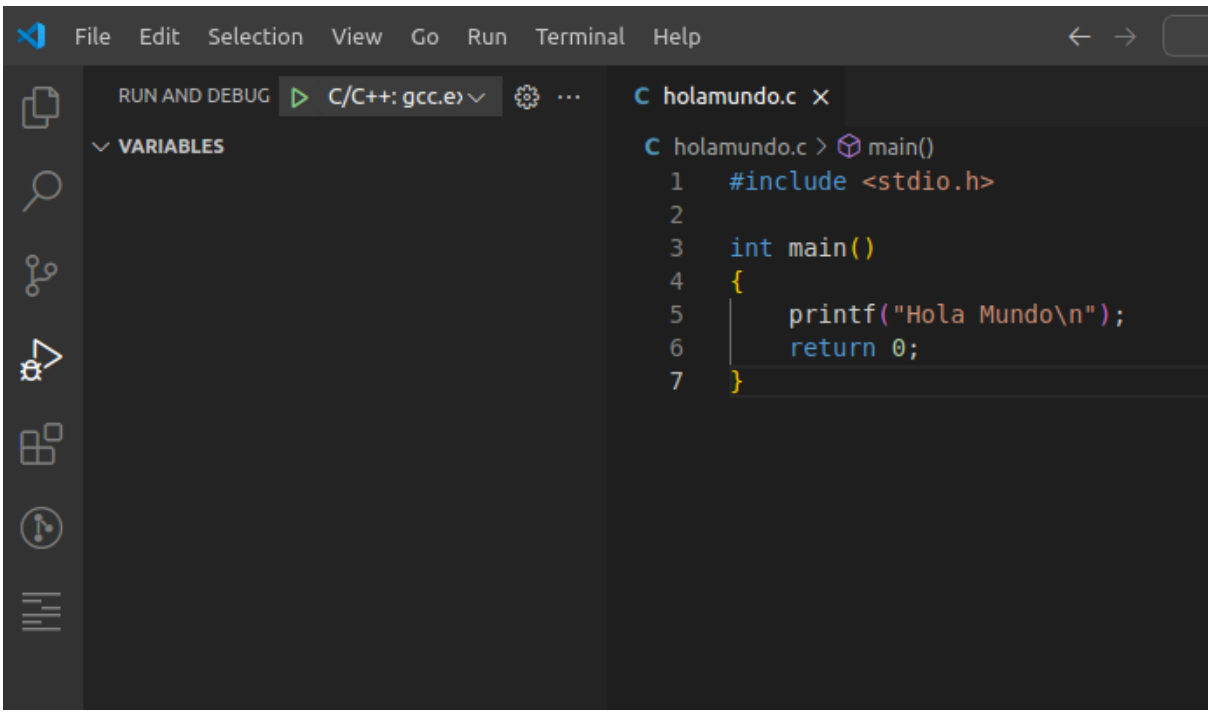
```
{
    "version": "0.2.0",
    "configurations": [
        {
            "name": "C build and debug active file",
            "type": "cppdbg",
            "request": "launch",
            "program": "${fileDirname}\\${fileBasenameNoExtension}.exe",
            "args": [],
            "stopAtEntry": false,
            "cwd": "${fileDirname}",
            "environment": [],
            "externalConsole": false,
            "MIMode": "gdb",
            "miDebuggerPath": "C:\\msys64\\ucrt64\\bin\\gdb.exe",
            "setupCommands": [
              {
                "description": "Enable pretty-printing for gdb",
                "text": "-enable-pretty-printing",
                "ignoreFailures": true
              },
              {
                "description": "Set Disassembly Flavor to Intel",
                "text": "-gdb-set disassembly-flavor intel",
                "ignoreFailures": true
              }
            ],
            "preLaunchTask": "C Build active file"
          }
    ]
}
```

Luego podremos crear nuestro archivo **.c** y ejecutarlo, para esto tenemos dos alternativas.

Opción 1 para ejecutar:
Con la opción “Run and Debug” del panel lateral izquierdo.

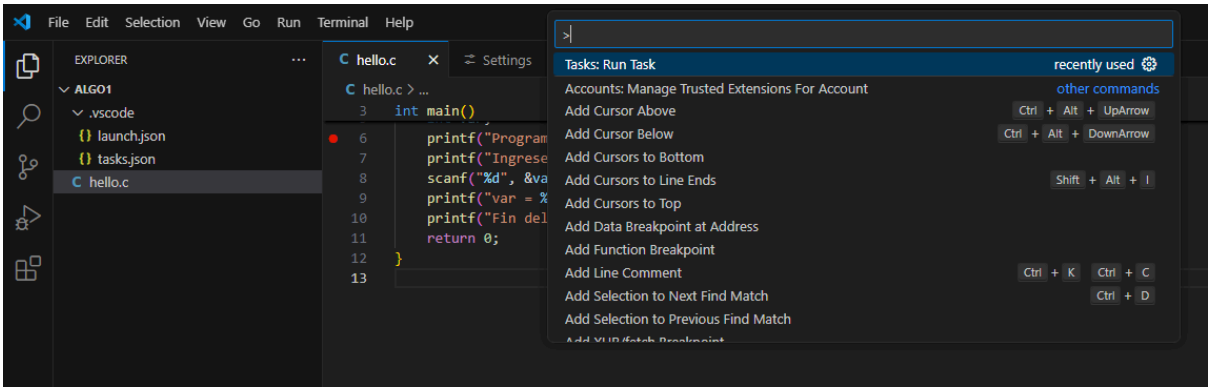
En el caso que la ejecución falle, se recomienda modificar el parámetro externalConsole de false a true:

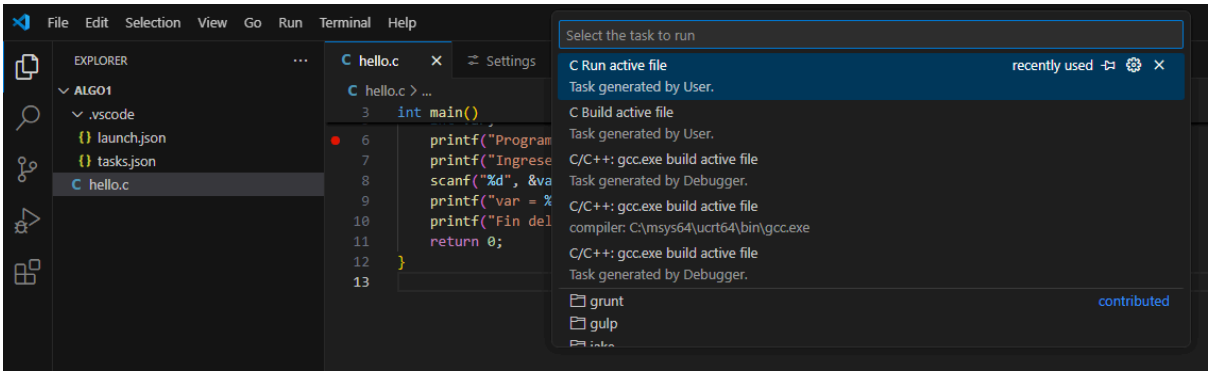
```
"externalConsole": true,
```



Opción 2 para ejecutar:

Haciendo uso de las tasks. Ejecutar **`ctrl+shift+p`** para desplegar el menú, seleccionar la opción **`Tasks: Run Task`**, aquí se desplegarán las tareas disponibles, primero ejecutaremos la tarea **`"C Build active file"`** para generar el ejecutable **`.exe`** y luego se puede ejecutar la cantidad de veces que se desee la tarea **`"C Run active file"`** para ejecutar el programa.





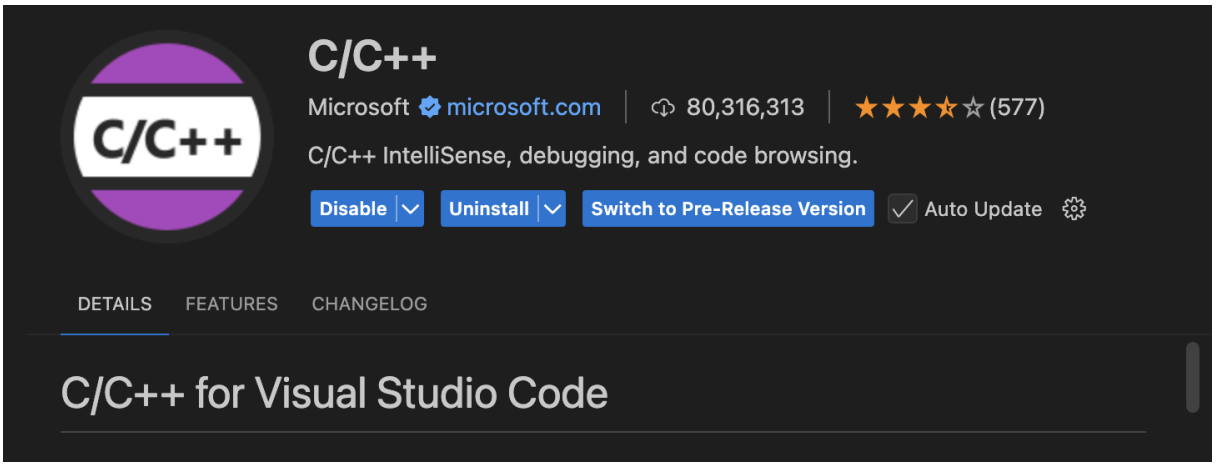
Entorno Linux

Guia configuración: https://code.visualstudio.com/docs/cpp/config-linux

Entorno Mac

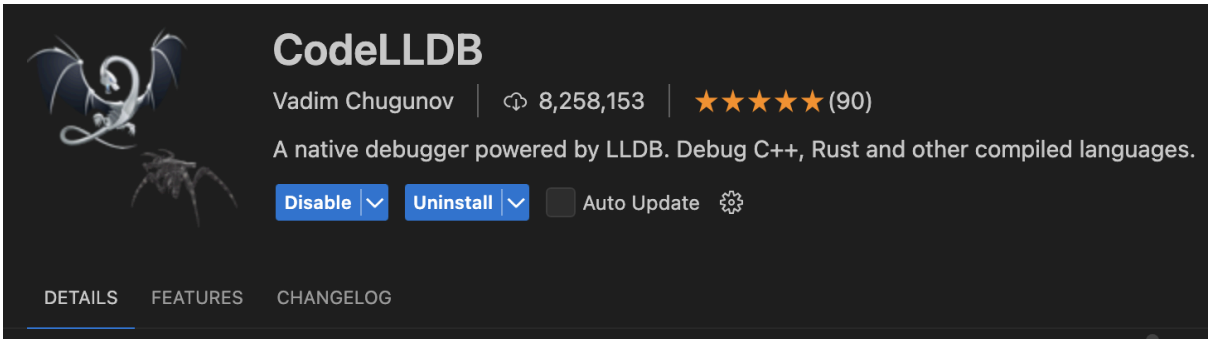
Una vez instalado VSCode se deben agregar los siguientes dos plugins:

1) **C/C++ for Visual Studio Code**
https://marketplace.visualstudio.com/items?itemName=ms-vscode.cpptools



2) **CodeLLDB**

https://marketplace.visualstudio.com/items?itemName=vadimcn.vscode-lldb

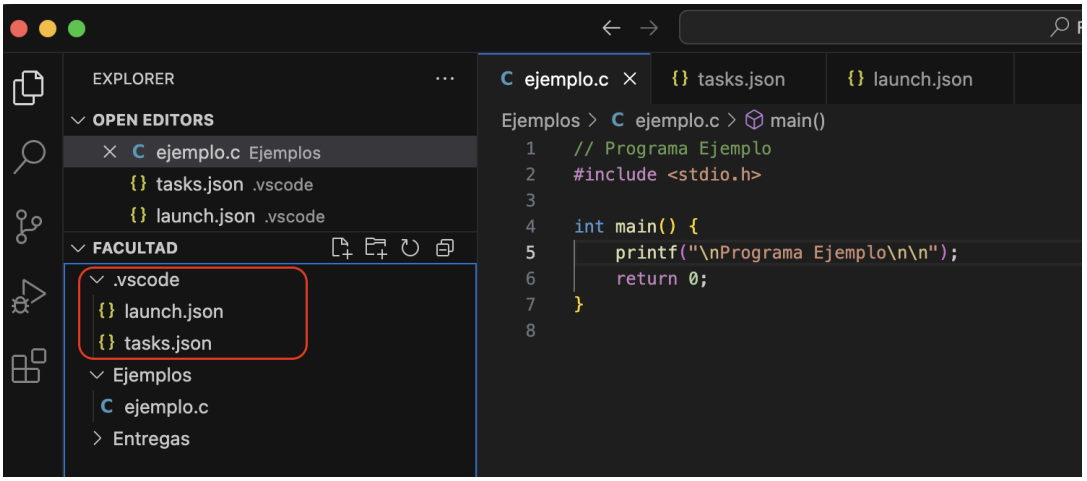


Luego se debe validar que se tenga el compilador **clang**, por línea de comandos ejecutamos: **clang –version**

```
diserra@AR000JC29VH6TC2 Facultad % clang --version
Apple clang version 16.0.0 (clang-1600.0.26.4)
Target: arm64-apple-darwin23.6.0
Thread model: posix
InstalledDir: /Applications/Xcode.app/Contents/Developer/Toolchains/XcodeDefault.xctoolchain/usr/bin
```

En caso de no disponer de este, se deberá instalarlo.

Dentro de nuestra carpeta de trabajo configuraremos dentro del directorio **.vscode** los archivos **launch.json** y **tasks.json**



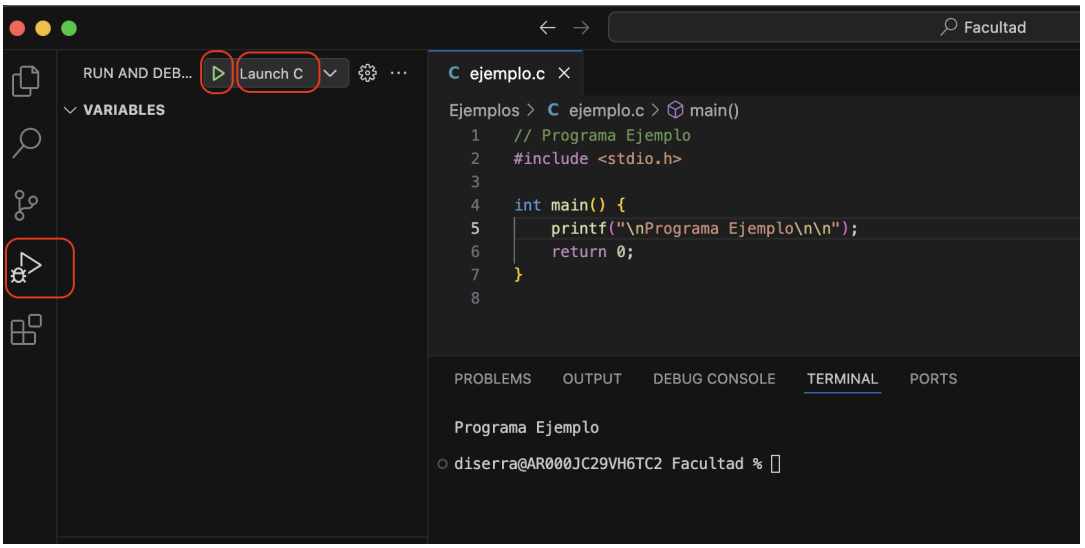
launch.json

```
{
    "version": "0.2.0",
    "configurations": [
        {
            "type": "lldb",
            "request": "launch",
            "name": "Launch C",
            "program": "${fileDirname}/${fileBasenameNoExtension}",
            "args": [],
            "cwd": "${fileDirname}",
            "preLaunchTask": "C/C++: clang compilar archivo activo"
        }
    ]
}
```

tasks.json

```
{
    "tasks": [
        {
            "type": "cppbuild",
            "label": "C/C++: clang compilar archivo activo",
            "command": "/usr/bin/clang",
            "args": [
                "-fcolor-diagnostics",
                "-fansi-escape-codes",
                "-g",
                "${file}",
                "-o",
                "${fileDirname}/${fileBasenameNoExtension}"
            ],
            "options": {
                "cwd": "${fileDirname}"
            },
            "problemMatcher": [
                "$gcc"
            ],
            "group": "build",
            "detail": "Tarea generada por el depurador."
        }
    ],
    "version": "2.0.0"
}
```

Ahora ya estamos en condiciones de ejecutar nuestro programa y realizar debug del mismo.
En el menú de debug se debe seleccionar la nombre de la configuración creada en el archivo **launch.json**



Entorno listo.

Debug en VSCode

https://code.visualstudio.com/docs/editor/debugging

Compilación y ejecución manual

En el caso de ser necesario podemos compilar nuestro programa manualmente mediante la ejecución de instrucciones por línea de comando en la consola.
Para esto debemos disponer de un compilador como puede ser **gcc.**

Abrimos una terminal y nos ubicamos en nuestro directorio de trabajo.

```
Símbolo del sistema

F:\Diego\Estudio\Facultad\Algo1>dir
 El volumen de la unidad F no tiene etiqueta.
 El número de serie del volumen es: F183-81D0

 Directorio de F:\Diego\Estudio\Facultad\Algo1

07/04/2025  07:30 p.m.    <DIR>          .
07/04/2025  07:30 p.m.    <DIR>          ..
07/04/2025  03:13 p.m.    <DIR>          .vscode
07/04/2025  07:17 p.m.               230 hello.c
               1 archivos            230 bytes
               3 dirs  46.164.840.448 bytes libres
```

Luego de identificar nuestro archivo de código fuente, en este caso **hello.c,** procederemos a ejecutar el comando de compilación y generación del archivo ejecutable.
El comando consta de los siguientes componentes:

```
gcc -g <nombre_archivo>.c -o <nombre_ejecutable>.exe
```

-g (parámetro opcional para agregar información de debug)
-o (parámetro para indicar el nombre de salida del archivo ejecutable)

Una vez ejecutado, vemos que se generó el archivo **hello.exe**, listo para ser ejecutado.

```
F:\Diego\Estudio\Facultad\Algo1>gcc -g hello.c -o hello.exe

F:\Diego\Estudio\Facultad\Algo1>dir
 El volumen de la unidad F no tiene etiqueta.
 El número de serie del volumen es: F183-81D0

 Directorio de F:\Diego\Estudio\Facultad\Algo1

07/04/2025  07:48 p.m.    <DIR>          .
07/04/2025  07:48 p.m.    <DIR>          ..
07/04/2025  03:13 p.m.    <DIR>          .vscode
07/04/2025  07:17 p.m.               230 hello.c
07/04/2025  07:48 p.m.           135.717 hello.exe
               2 archivos        135.947 bytes
               3 dirs  46.164.701.184 bytes libres
```

Luego podemos ejecutar el programa simplemente escribiendo el nombre del archivo ejecutable.

```
F:\Diego\Estudio\Facultad\Algo1>hello.exe
Programa de prueba
Ingrese numero: 565
var = 565
Fin del programa

F:\Diego\Estudio\Facultad\Algo1>
```